PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers

***Abstract*—We present PORTER, a plan-committed router for multi-core distributed quantum computers (DQCs). Where TELESABRE and DSABRE score every inter-core candidate together with its staging SWAPs in one flat action pool, PORTER separates the two decisions: it first scores every legal remote *plan* on its physical and lookahead merits alone, commits to the best one, and only then generates the preparation SWAPs specific to that plan, so preparation effort is never spent on a plan that is not selected. PORTER also unifies the three inter-core communication primitives used in prior work — state teleportation (TELEDATA), single-gate teleportation (TELEGATE), and multi-gate cat-entangler packets (TELEGATEPACKET) — under this one decision loop. At each iteration the router drains directly executable gates, resolves intra-core routing with an extended-set lookahead SWAP score, and, only when the DAG front contains solely inter-core gates, scores every legal remote plan (teleport, telegate, and same-control packet candidates together with capacity-blocked variants) *before* committing to any core-preparation SWAPs. A breadth-first capacity-relief search evacuates the nearest reservoir core when the best remote plan is blocked by insufficient free capacity, and an opportunistic-relief comparator lets a severely blocked plan pre-empt a merely-good normal plan when the margin is large enough. We audit the reference implementation with an independent replay verifier across 60 randomized circuits and the full MQT-Bench evaluation suite, apply a set of behaviour-preserving algorithmic-complexity fixes that yield a $4\times$ compile-time speedup with byte-identical output, and evaluate against TELESABRE and DSABRE on the same physical topologies at 25, 64, and 80 logical qubits, plus a third initial-layout source generated by `pytket-dqc`'s hypergraph partitioner. In the process we identify and patch a latent out-of-bounds bug in TELESABRE's reference C implementation that crashes it on small-core architectures, and a filename-handling bug in its argument parser that had caused an earlier version of this evaluation to under-report its convergence. Under each router's own best-practice initial-layout protocol, PORTER reduces geometric-mean EPR consumption by 42.1% against TELESABRE on the 25-qubit suite and by 16.7% on the 64-qubit suite; it trails DSABRE under this protocol, particularly on sparse and chain-structured circuits. Feeding both DSABRE and PORTER TELESABRE's own initial layout isolates routing quality from layout quality: on the 25-qubit suite this reverses the ranking (-40.1% vs. TELESABRE for PORTER, -37.3% for DSABRE), indicating that part of DSABRE's own-protocol advantage there traces to its SabreLayout-based initial allocation rather than to routing-time decisions alone; on the 64-qubit suite DSABRE keeps the lead under the shared layout too (-47.9% vs. -37.8%), so the layout-versus-routing split of that advantage is architecture-dependent. At larger scale, PORTER shows a distinct advantage in completing circuits the other two cannot: on an 80-qubit, 23,381-CX random circuit, both TELESABRE and DSABRE exceed our uniform 900 s per-call compiler timeout while PORTER completes in 40 s at 10,855 EPR pairs. Running DSABRE past that cap shows it is not deadlocked: it places 7,713 EPR pairs and 41,421 SWAPs, genuine forward progress, before exhausting its own 20,000-iteration budget (carried over, unincreased, from the 64-qubit suite) on its first of up to nine required routing passes – a resource-budget limitation on this unusually dense circuit rather than a deadlock. Code and raw results are available in the accompanying repository.**

***Index Terms*—distributed quantum computing, circuit routing, qubit teleportation, plan-committed routing, EPR minimisation, qubit mapping, lookahead heuristic**

I. INTRODUCTION

Distributed quantum computers (DQCs) interconnect several small, high-fidelity quantum processing units (QPUs, or *cores*) with inter-core links that support remote entanglement generation [1], offering a near-term path to scale beyond what a single monolithic chip can host. Compiling a circuit onto a DQC decomposes into three coupled sub-problems [2]: *(i)* statically assigning logical qubits to cores, *(ii)* choosing, for each non-local two-qubit gate, which communication primitive realises it, and *(iii)* routing and scheduling the resulting sequence of intra-core SWAPs and inter-core operations subject to communication-port capacity. Every communication primitive consumes one or more EPR pairs, and EPR generation is slow, noisy, and rate-limited on current hardware [3], [4], making EPR minimisation the dominant compilation objective.

PORTER addresses stage (iii): given a fixed initial layout, it orders intra-core SWAPs and inter-core operations to minimise EPR consumption. Unlike DSABRE [5], which restricts itself to state teleportation (TELEDATA), PORTER follows TELESABRE [6] in also supporting gate teleportation (TELEGATE) via the cat-entangler protocol [7], [8], and extends it with a same-control *packet* primitive (TELEGATEPACKET) that amortises one EPR pair over an ordered run of consecutive remote CX gates sharing a control qubit, in the spirit of gate-sequence amortisation used by AutoComm [9].

The router departs from TELESABRE and DSABRE in *when* it commits to core-preparation SWAPs. Both score inter-core candidates and their staging SWAPs together in one flat action pool. PORTER instead separates the decision into two phases: first, every legal remote *plan* — a teleport direction, a single telegate, a same-control packet, or a capacity-blocked variant of any of these — is scored using only its physical and DAG-lookahead properties, independent of how many preparation SWAPs it would take to make it physically legal right now; only *after* one plan is selected does the router generate and apply preparation SWAPs specific to that plan, repeating until the plan becomes physically ready or no further preparation SWAP improves it. This plan-first ordering means preparation effort is never spent scoring or partially staging a remote action that a better one will preempt.

Contributions.

- **A plan-first remote-action loop** that separates candidate scoring from preparation-SWAP generation (Section III-C), unifying teleport, telegate, and same-control

packet primitives under one direct/latent/lookahead gain decomposition (Section III-D).
- **A two-tier capacity-relief mechanism**: a breadth-first search for the nearest reservoir core when a plan is capacity-blocked (Section III-G), plus an opportunistic comparator that lets a severely blocked candidate preempt a merely-good normal one.
- **An independent replay verifier** (Section IV) that re-executes the emitted operation stream against its own physical-layout simulation and checks gate coverage, DAG-order faithfulness, and every physical legality constraint (coupling-edge adjacency, port occupancy, cat-section isolation), used to certify correctness across 60 randomized circuits and the full evaluation suite.
- **A set of behaviour-preserving performance fixes** — incremental DAG-frontier maintenance, copy-on-write DAG-simulation snapshots, $O(1)$ free-slot accounting, and apply/restore local-SWAP scoring in place of layout cloning — that cut compile time $4\times$ on our suite with byte-identical output, verified by hashing the serialized operation stream before and after (Section V).
- **A head-to-head evaluation** against TELESABRE and DSABRE on identical physical topologies (Section VI), in the course of which we discovered and patched a latent out-of-bounds write in TELESABRE's reference C implementation that segfaults on cores smaller than its original 4×4 design point (Section VI-D).

II. BACKGROUND

A. Quantum Circuits and Routing

A quantum circuit acts on n logical qubits through a sequence of gates; we work in the restricted basis $\{\texttt{id}, \texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$ used by MQT-Bench's Qiskim-opt3-transpiled circuits, i.e. arbitrary one-qubit (1Q) gates plus the two-qubit (2Q) CX gate. A circuit is represented as a directed acyclic graph (DAG): for every logical qubit, consecutive gates acting on it are connected by a dependency edge. The *front* (or *DAG front*) is the set of gates with no unexecuted predecessor; only front gates can execute or be routed towards execution at a given step.

Physical qubits are partitioned into *cores*; each core has an intra-core coupling graph, and pairs of cores may be connected by an inter-core link whose two endpoints are designated *communication ports*. A *layout* maps every logical qubit to a physical position. A 2Q gate is *local* if both operands share a core and *remote* otherwise. Routing a local gate that is not yet adjacent requires intra-core SWAPs; routing a remote gate requires an inter-core communication primitive.

B. Communication Primitives

State teleportation (TELEDATA). Consumes one EPR pair to move a logical qubit's state from a physical position adjacent to a free source-side communication port to the (necessarily free) destination-side port in a neighbouring core [10]. The qubit then participates in local gates in its new core.

Gate teleportation (TELEGATE). Executes a single remote CX directly across two adjacent cores via a cat-entangler protocol [7], [8]: one EPR pair entangles the two communication ports, the remote CX is applied using the target-side port as a proxy for the control qubit, and the entanglement is consumed on disentangling. Neither logical qubit physically moves.

Same-control packets (TELEGATEPACKET). When several consecutive remote CX gates in the DAG front share the same control qubit and land in the same target core, PORTER amortises them onto a *single* cat-entangler resource: the entangling and disentangling steps bracket an ordered sequence of remote CX and target-core 1Q gates, all executed while the two communication ports stay reserved. This is the same amortisation idea used by AutoComm [9], specialised to the shared-control case that arises naturally at the DAG front (Section II-A).

C. SABRE

SABRE [11] routes a single-core circuit by repeatedly selecting the SWAP that minimises a front-layer distance term plus a weighted extended-set lookahead term, both expressed as Δ (old distance minus new distance) on gates sharing a qubit with the candidate SWAP. PORTER inherits this scoring form for its intra-core stage (Section III-B) and extends the same direct/lookahead decomposition to its inter-core plan scorer (Section III-D).

III. THE PORTER ROUTER

PORTER is implemented as `PorterRouter.compile` in the `porter` package (3,148 lines across `hardware.py`/`layout.py`/`model.py`/`router.py`, the latter holding the routing loop itself at 2,519 lines). Given a circuit and an initial layout, it repeats the following loop (Algorithm 1) until every gate has been emitted or a configurable iteration budget is exhausted.

A. Front Classification

Every gate g in the (size-capped) DAG front is classified under the current layout ℓ: 1Q gates are always directly executable; a CX gate $g = (q_i, q_j)$ with $\kappa_\ell(q_i) = \kappa_\ell(q_j) = c$ (same core) is *direct-local* if $d_{\mathrm{L}}(c, \ell(q_i), \ell(q_j)) = 1$ and *local-needing-SWAP* otherwise; a CX gate with $\kappa_\ell(q_i) \neq \kappa_\ell(q_j)$ is *adjacent-remote* if the two cores are core-graph neighbours and *non-adjacent-remote* otherwise. Directly executable gates are always drained first and completely, so remote-stage decisions are only made once the front contains solely gates that need routing.

B. Intra-Core SWAP Scoring

For a local-needing-SWAP gate set F_{loc}, PORTER builds candidate SWAP edges from the coupling-graph neighbours of every physical position touched by F_{loc}, simulates each SWAP, and scores it as

$$J(a) = \bar{d}_{\mathcal{F}}(a) + \lambda_{\mathrm{loc}}\, \bar{d}_{\mathcal{E}}(a), \qquad \lambda_{\mathrm{loc}} = 0.8, \tag{1}$$

Algorithm 1 PORTER `compile`(C, ℓ_0)

$\ell \leftarrow \ell_0$; initialise DAG active/frontier sets from C
while DAG has active gates **do**
  $F \leftarrow$ DAG front (bounded by `dag_front_limit`)
  classify F into singles, direct-local CX, local-CX-needing-SWAP, adjacent-remote CX, non-adjacent-remote CX *(Sec. III-A)*
  if singles or direct-local CX present **then**
    execute them all; remove from DAG; **continue**
  end if
  if local-CX-needing-SWAP present **then**
    generate candidate intra-core SWAPs, apply the best-scoring one (Eq. 1); **continue** *(Sec. III-B)*
  end if
  generate normal remote plans $\mathcal{P}$ and blocked-teleport plans $\mathcal{B}$ *(Sec. III-D)*
  if $\mathcal{P} \neq \emptyset$ **then**
    $p^* \leftarrow$ best ready-or-preparable plan in $\mathcal{P}$
    if some $b \in \mathcal{B}$ has gain $\gg$ gain(p^*) **then**
      $p^* \leftarrow$ best opportunistic capacity-relief plan for b *(Sec. III-G)*
    end if
  else
    BFS from each blocked target core to its nearest reservoir; $p^* \leftarrow$ best tail-relief plan *(Sec. III-G)*
  end if
  if p^* is `None` **then**
    report deadlock; abort
  end if
  apply p^*'s own preparation SWAPs until ready, then execute it
end while
return emitted operations, final layout, stats

the mean post-SWAP local distance over F_{loc} plus a weighted mean over an extended set $\mathcal{E}_{\text{loc}}$: up to N_{loc}=20 local CX gates collected from the next L_{loc}=12 DAG layers via a virtual frontier simulation (Section III-H). Ties are broken first by the raw front-layer distance, then, if still tied, by a decision uniformly sampled from a seeded random-number generator, with every seeded tie-break recorded in `RouterStats` for reproducibility auditing.

C. Plan-First Remote Stage

When the front contains only remote CX gates, PORTER generates two disjoint candidate sets from the current layout:

- **Normal plans** $\mathcal{P}$: for every adjacent-remote CX gate, a single TELEGATE; if a same-control run of ≥ 2 remote CX gates can be collected by simulating the DAG front forward (Section III-E), a TELEGATEPACKET; and, for either endpoint, a TELEDATA plan into the partner's core *if* that core already has at least `min_free_before_teleport` ($f_{\min}$=2) free physical positions. For non-adjacent-remote CX gates, only distance-reducing TELEDATA hops (one core-graph step closer to the gate's other endpoint) with sufficient destination capacity are generated.
- **Blocked plans** $\mathcal{B}$: the same directional TELEDATA candidates, but for destination cores that currently fail the $f_{\min}$ capacity test. These are never selected directly; they exist to detect when *no* normal plan exists because every candidate destination is saturated, or when a normal plan is technically available but a severely blocked plan promises much higher gain if capacity is relieved first.

Each plan is scored (Section III-D) *before* any preparation SWAP for it is generated. The router then:

1) selects the best plan in $\mathcal{P}$ that is either already physically ready or admits at least one improving preparation SWAP right now (Section III-F);
2) compares its gain against the best plan in $\mathcal{B}$; if the blocked plan's gain exceeds the selected normal plan's by more than a fixed margin m_{assist}=5.0, the router instead searches for an *opportunistic capacity-relief* plan for the blocked plan's saturated destination core (Section III-G) and, if one exists, executes that instead;
3) if $\mathcal{P} = \emptyset$ (every remote CX gate is non-adjacent), the router runs the same breadth-first capacity-relief search directly on $\mathcal{B}$'s destination cores, grouped by hop count, taking the shortest-hop group with a viable candidate.

Once a plan is committed, the router repeatedly generates and applies *only that plan's* preparation SWAPs (Section III-F) until it becomes ready, no further improving preparation SWAP exists, or the full decision state (plan identity, layout, RNG state) repeats — the latter raising a hard error rather than looping forever, bounded by `max_prep_iterations_per_plan` (10^4).

D. Plan Scoring: Direct, Latent, and Lookahead Gain

Every plan's score is $\text{total} = \text{direct} + \text{lookahead}$, computed differently per plan kind but sharing one Δ-form convention: for a CX gate $g = (q_i, q_j)$ and a candidate qubit relocation, $\Delta(g) = d_{\text{C}}(\kappa(q_i), \kappa(q_j)) - d_{\text{C}}(\kappa'(q_i), \kappa'(q_j))$, the reduction in core-graph distance between g's two cores.

TELEGATE contributes a constant direct gain of 1 (one remote CX resolved) and no lookahead term.

TELEGATEPACKET contributes a direct gain equal to the number of remote CX gates in the collected packet – every gate in an accepted packet is current, not speculative, work, since the whole packet executes under one cat-entangler resource once started.

TELEDATA decomposes its influence into three tiers, all computed by simulating the DAG frontier under the hypothetical post-teleport core assignment (Section III-H):

- *Direct*: front-layer CX gates already involving the moving qubit, plus the seed gate itself.
- *Latent-direct*: CX gates on the moving qubit that appear in the simulated DAG front while the simulator can still make virtual progress with only 1Q gates and now-local CX gates (i.e. gates whose post-teleport cores coincide); these gates are current pressure, not prediction, because the candidate teleport directly determines their post-move core distance the moment they surface.

- *Lookahead*: CX gates on the moving qubit found in up to L_{tp}=16 further simulated DAG layers (capped at N_{tp}=50 gates) beyond the point where the latent-direct simulation naturally stops (no further gate is virtually executable). Layer k's contribution is weighted by $\max(w_{\min}, w_0\gamma^{k-1})$ with w_0=0.9, γ=0.9, $w_{\min}$=0.5.

$$\text{direct} = \sum_{g\in\text{direct}} \Delta(g) + w_{\text{ld}} \sum_{g\in\text{latent}} \Delta(g), \tag{2}$$

$$\text{lookahead} = \sum_{(g,k)\in\text{la}} w_k\,\Delta(g), \tag{3}$$

with w_{ld}=1.0 (`teleport_latent_direct_weight`). A capacity-risk term $1/\max(1, f_{\text{dst}}-1)$ (post-move free count in the destination core) is tracked alongside the score for use as a tie-break dimension but does not enter total directly; plans are primarily ranked by total gain (descending), then readiness, then preparation cost, then plan kind (TELEDATA $\succ$ TELEGATEPACKET $\succ$ TELEGATE), then capacity risk, then lookahead gain, with a fully deterministic stable key as the final tie-break before any seeded random choice.

E. Same-Control Packet Collection

Starting from a seed adjacent-remote CX gate $g = (q_c, q_t)$ with control core c_c and target core c_t, PORTER simulates the DAG frontier forward for up to `extended_packet_window` (120) layers, greedily absorbing into the packet: (i) any front CX gate sharing control q_c whose target is also in c_t, up to `max_packet_size` (8) such gates; and (ii) any front 1Q gate whose qubit is already in c_t, since these execute freely while the packet's cat resource is live. Collection stops at the first layer that contributes nothing, or once the size cap is reached. Because this DAG-frontier simulation admits gates from later generations that are not literally in the (bounded) current DAG front, a packet may contain gates unreachable by a naively front-limited scan; the resulting execution order is preserved and replayed exactly at commit time (Section III-I).

F. Preparation SWAPs

A committed plan's readiness requirements are exactly two *(core, neighbour-core, endpoint)* triples (source and destination for TELEDATA; control and target for TELEGATE/ TELEGATEPACKET, with TELEGATEPACKET's target-side routing deferred to execution time since each packet member routes independently). For each unmet requirement, candidate preparation SWAPs are restricted to edges incident to either the plan's logical endpoint or a nearest free physical position tied for minimum distance to the relevant communication port (only while that port remains occupied); each candidate is scored by the post-SWAP value of the same readiness-cost function used to rank blocked plans (Section III-D), and the router applies at most one improving SWAP per core side per pass, repeating until the plan is ready.

G. Capacity Relief

Tail-relief BFS. When every remote plan is capacity-blocked, PORTER runs a breadth-first search from each blocked destination core to the nearest *reservoir* core – one with at least $f_{\min}$ free slots – optionally chained beyond one hop (`capacity_relief_chain_enabled`). Every logical qubit currently in the BFS path's penultimate core becomes a tail-relief TELEDATA candidate into the reservoir, scored and selected by the same plan machinery, grouped and tried in ascending hop-count order.

Opportunistic relief. Even when a normal plan is available, a severely blocked plan can still win if relieving its target core's capacity would unlock substantially more gain (Section III-C); this differs from tail-relief BFS in firing preemptively, before any plan is actually starved, whenever the margin m_{assist} is exceeded.

H. Virtual DAG-Frontier Simulation

Extended-set collection (Sections III-B–III-E) and teleport lookahead (Section III-D) share one primitive, `DagFrontierSimulator`: a copy-on-write view over the live DAG state that tracks a private *overlay* of predecessor-count decrements against the live `active` and `pred_base` maps (shared read-only), so that snapshotting costs $O(|\text{frontier}|)$ rather than $O(|\text{remaining circuit}|)$. Removing a batch of gate IDs from a snapshot updates the overlay and exposes any successor whose overlay count reaches zero as newly virtual-frontier. Because the overlay is private to one snapshot, several independent virtual simulations can run from the same live DAG state without interfering (Section V discusses the performance impact of this design versus a copying implementation).

I. Applying Committed Operations

Local SWAPs and CX gates update the layout in place. Teleports move a logical qubit's physical/core mapping and refresh the affected cores' free-slot queues. Single telegates and packets emit a `cat_entangler` operation (consuming one EPR pair, marking the two communication ports live), followed by one `cat_cx` per remote CX (packets additionally route the target qubit to its communication port with ordinary local SWAPs, forbidden from touching the two live ports), and close with a `cat_disentangler`. Final circuit depth is computed by a physical-qubit timeline pass matching TELESABRE's convention: 1 time unit per local operation, 5 per teleport or (dis)entangler pair, applied per touched physical qubit and taking the maximum over all qubits.

J. Computational Complexity

Let N be the circuit's gate count, n its logical-qubit count, K the number of cores, and M the per-core physical-qubit count (P=KM total physical qubits). Hardware distance tables (d_{L} per-core, d_{C} core-graph) are precomputed once at construction, in $O(KM^2 \log M + K^2 \log K)$, matching the standard two-level Dijkstra decomposition used by comparable routers [5]. Per iteration, front classification and direct-execution draining cost $O(|F|)$; intra-core SWAP scoring

evaluates $O(M)$ candidate edges each at $O(|F_{\text{loc}}| + N_{\text{loc}})$ via one $O(|\text{frontier}|)$-cost DAG snapshot, reused across candidates; remote-plan scoring evaluates $O(n)$ candidates each at $O(N_{\text{tp}})$ via the same amortised snapshot cost. Free-slot-queue maintenance is $O(1)$ amortised per layout mutation since only the touched core(s) are refreshed (Section V). Summing over N iterations gives $T_{\text{total}} = O\big(N(Mn + KM^2 \log M)\big)$ excluding the one-time precomputation term, in line with the per-iteration cost of other core-local distributed routers [5], [6].

IV. Correctness Verification

Beyond code review, we built an independent replay verifier (`verify_distributedrouter.py`) that treats the router as a black box: it takes only the emitted `CompiledOp` stream, the original circuit, and the initial layout, and re-derives, from scratch, whether the stream is a faithful and physically legal execution. The verifier maintains its own physical-position map and a per-logical-qubit FIFO queue of pending gate IDs (derived directly from the circuit, independent of any router-internal DAG state), and checks, per emitted operation:

- every gate is popped from the head of *every* one of its operand qubits' queues before being marked executed, so gates cannot execute out of DAG order or twice;
- every `local_swap`/`local_cx` acts on a genuine intra-core coupling edge, at the physical positions the verifier's own simulated layout says the operand logical qubits occupy;
- every `teleport` uses the correct inter-core port pair for its two cores, both ports were free beforehand, and the source qubit was coupling-adjacent to the source port;
- every `cat_entangler`, `cat_cx`, and `cat_disentangler` triple observes port-adjacency and free-port preconditions, and no other operation touches either communication port while a cat section is open;
- the emitted `RouterStats` counters (EPR, teleport, telegate, packet, SWAP, local-CX, single counts) match the counts the verifier itself derives from the op stream, catching any bookkeeping drift between what is emitted and what is counted.

We ran the verifier against 60 randomized circuits (4–40 logical qubits, 20–600 gates, both architectures, seeds 0–59) and against every circuit in the evaluation suite (Section VI); all pass. An earlier version of the verifier surfaced a genuine accounting bug in our own harness – packet telegates were being double-counted against the single-telegate stat – which was fixed before any performance work began, and every subsequent optimisation (Section V) was re-verified end to end.

V. Performance Engineering

Profiling the unoptimised router on the 64-qubit suite (Section VI) attributed the majority of wall-clock time to two sources: re-copying the entire remaining-circuit DAG state on every extended-set or teleport-lookahead snapshot (∼55% of total time on the deepest circuit), and re-scanning `state.remaining` for the zero-predecessor front on every iteration (∼13%). We applied five behaviour-preserving changes:

1) **Copy-on-write DAG snapshots** (Section III-H): a snapshot shares the live active-gate set and predecessor counts read-only and only writes to a private overlay, so its cost scales with the size of the frontier being explored rather than the number of gates left in the whole circuit.
2) **Incremental DAG-front maintenance**: the router now maintains `RouterState.dag_frontier` as gates are removed, rather than recomputing it by filtering the full remaining-gate list every iteration; `state.remaining` itself is now synced lazily, only at the three points where the caller actually reads it.
3) **Cached hardware distance/adjacency tables**: all-pairs core distances, sorted core-neighbour lists, and sorted local-neighbour lists are computed once at construction instead of via a fresh BFS on every `core_distance`/`core_neighbors` call.
4) $O(1)$ **free-slot accounting**: `Layout.free_count` now reads a per-core occupancy counter maintained incrementally by the two layout-mutation entry points, instead of re-scanning every physical qubit in the core; free-slot-queue refresh after a layout change is restricted to the one or two cores actually touched.
5) **Apply/restore SWAP scoring**: candidate-SWAP scoring loops (intra-core SABRE scoring, Section III-B; preparation-SWAP scoring, Section III-F) previously cloned the entire layout per candidate; they now apply the SWAP, score, and restore the exact pre-swap mapping in place.

We verified behaviour preservation by hashing the fully serialized, sorted-key JSON operation stream (SHA-256) for eleven representative circuit/architecture/seed combinations before and after every change, together with the `seeded_tie_break_count` statistic (an exact proxy for RNG consumption): all eleven hashes and tie-break counts are identical pre- and post-optimisation, and the independent replay verifier (Section IV) passes on both. Total wall time on that suite fell from 6.24 s to 1.55 s (4.0×); the 64-qubit `ae` circuit alone fell from 2.77 s to 0.56 s, and a 13,126-CX `qnn` stress circuit fell from ≥6.2 s (measured with only the copy-on-write snapshot change reverted, as a lower bound) to 1.9 s.

VI. Experimental Evaluation

A. Setup

Architectures. We use the same two grid-of-grids topologies introduced by TeleSABRE [6] and re-used by DSABRE [5]: architecture A, a 2×2 array of 3×3 cores (36 physical qubits, 4 inter-core links), and architecture H, a 2×3 array of 4×4 cores (96 physical qubits, 7 inter-core links). Both are instantiated identically for all three routers: we build PORTER's `Hardware` object once and convert it into a DSABRE `DistributedArchitecture` with matching physical-qubit IDs, intra-core edges, and inter-core ports (verified programmatically before every run), and TeleSABRE

TABLE I
BENCHMARK CIRCUIT CX COUNTS (MEASUREMENTS/BARRIERS STRIPPED).

Circuit	25q (arch. A)	64q (arch. H)	80q (arch. H)
AE	558	1962	—
GHZ	24	63	79
Graphstate	25	64	80
QFT	580	1966	2660
QNN	1223	8126	79
Random	1124	1627	23381

is run against the project's own `A_grid_2_2_3_3.json` / `H_grid_2_3_4_4.json` device files, whose inter-core edges we confirm match exactly.

Benchmarks. We use the same MQT-Bench [12] circuit files DSABRE's own evaluation uses (confirmed by exact CX-count match against its published tables): a 25-qubit suite (AE, GHZ, Graphstate, QFT, QNN, Random) on architecture A, and a 64-qubit suite (the same six families) on architecture H, with QNN using the deeper 8,126-CX variant. Table I lists CX counts.

Third suite (80-qubit). To test at intermediate scale on the same architecture H, we generated an 80-qubit suite with `mqt.bench` 2.2.2 (device-independent generation followed by transpilation to our basis, since the installed `mqt.bench`'s target-aware native-gate path raised an unrelated `qiskit` compatibility error on every circuit we tried). AE is excluded: its phase-estimation construction hits a numerical-precision failure (`"Input matrix is not unitary"`) at this size in the installed `mqt.bench`/`qiskit` combination, independent of our code path; a reduced circuit-family set at larger qubit counts is already precedented by DSABRE's own 100q/200q/360q suites [5]. The generator's current QNN benchmark has no depth parameter and produces a near-linear circuit at this size (CX $\approx$ qubit count, Table I), so it should be read as structurally closer to GHZ than to the deeper QNN instances at 25q/64q.

Baselines. We compare against TELESABRE [6] and DSABRE [5]. For DSABRE we report only `dSABRE_BurstExt`, its BFS-layer extended-set variant and the project's own default/production router (its topological-extended-set ablation, DSABRE-Topo, is omitted).

Compiler timeout. Every individual compiler invocation – one TELESABRE subprocess run, one DSABRE routing-pass call, or one PORTER `compile` call – is capped at 900 s wall-clock; a call that exceeds it is recorded the same way as any other failure to complete (missing entry, excluded from that protocol's geometric mean). This is a harness-level bound applied uniformly across all three routers and all three protocols below, not specific to any one circuit.

Own-protocol comparison. Each router is first evaluated under its own established best-practice initial-layout protocol, best-of-three, matching the convention already used to compare TELESABRE and DSABRE to each other [5]:

- **TELESABRE**: the compiled C binary's built-in hungarian initial-layout heuristic, config seeds 0–2, best EPR count of the successful runs.
- **DSABRE**: Qiskit `SabreLayout` on the corners-removed architecture graph (seeds 0–2) followed by SABRE-style forward $\rightarrow$ backward $\rightarrow$ forward routing passes, best EPR count.
- **PORTER**: two capacity-respecting initial-layout heuristics, each run at three seeds (six candidates total), best EPR count. The first is `try15`, a balanced-first-layer placement that co-locates the qubits of each first-CX-layer gate and fills the rest by remaining core capacity; because its core assignment is deterministic given the circuit (only intra-core physical placement is seed-dependent), we add a second, genuinely seed-diverse candidate: a randomized interaction-graph-greedy placement that processes logical qubits in a seeded random order, placing each next to its heaviest already-placed CX neighbour when that neighbour's core still has capacity headroom above `min_free_before_teleport`. This second heuristic matters specifically on long qubit-chain circuits (GHZ): `try15` only ever co-locates the very first CX pair and scatters the rest uniformly at random, since GHZ's circuit structure never exposes a second non-overlapping first-layer CX pair to its greedy scan; on `ghz_25` this alone is the difference between 18 and 6 EPR pairs for PORTER (Table III).

We do not reuse DSABRE's SabreLayout-corners-removed candidates for PORTER in this comparison. DSABRE's corner-removal selects the four lowest-degree vertices of the *whole-chip* graph and excludes them from the SabreLayout coupling map, guaranteeing they stay free; on both of our architectures, whose physical-qubit ids are assigned core-by-core, these four vertices are, by construction, the four corners of a single core (core 0 in our numbering) – verified programmatically for both architectures we test. The other cores receive no guaranteed free slots from this mechanism at all, and on this suite it regularly leaves them with zero free physical qubits – not a valid starting state for PORTER's teleport mechanism, which requires `min_free_before_teleport` free slots in a core before it can ever be used as a teleport destination. This is a structural precondition mismatch between the two systems' initial-layout assumptions, not a routing-quality question. PORTER now includes a general safeguard against exactly this precondition – evicting one logical qubit from any core an externally supplied layout leaves fully packed, into the core with the most spare capacity, before routing begins (used below in the pytket-mapping comparison) – but we still exclude these candidates here to keep each router's own-protocol result tied to its own established best-practice initial-layout heuristic, not one designed for a different system.

Shared-layout comparison. To isolate routing quality from initial-layout quality, we additionally route both DSABRE and PORTER from TELESABRE's *own* initial layout: for the winning seed of the run above, we extract, from TELESABRE's optional JSON trace, the physical-to-logical mapping recorded at iteration 0 – i.e. the layout TELESABRE itself settled on after its internal forward/backward/forward layout refinement, immediately before its accepted forward

TABLE II
HARDWARE COST AND HEURISTIC PARAMETERS (SEE SECTION III).

Parameter	Symbol	Value
Min. free before TELEDATA	$f_{\min}$	2
Opportunistic-relief margin	m_{assist}	5.0
Local-SWAP extended-set weight	λ_{loc}	0.8
Local extended-set max layers	L_{loc}	12
Local extended-set max CX	N_{loc}	20
Packet collection window (layers)	–	120
Max packet size (remote CX)	–	8
TeleData lookahead max layers	L_{tp}	16
TeleData lookahead max CX	N_{tp}	50
Lookahead weight / decay / floor	$w_0, \gamma, w_{\min}$	0.9, 0.9, 0.5
Latent-direct weight	w_{ld}	1.0

routing pass – and feed that identical mapping, unmodified, to a single DSABRE run and a single PORTER run (no per-router layout search; the layout is now fixed and given). Unlike DSABRE's own SabreLayout, TELESABRE's hungarian heuristic reserves free-slot headroom per core by construction (tuned to its own teleport threshold), so it is compatible with PORTER's teleport precondition without adjustment on every circuit we tested.

Pytket-mapping comparison. As a third, independent layout source, we also route DSABRE and PORTER from a placement generated by `pytket-dqc`'s `HypergraphPartitioning` allocator [13] (best-of-3 KaHyPar seeds on a `NISQNetwork` matching our core graph), converted to a physical layout by the same topology-aware intra-core assignment DSABRE's own pytket benchmark uses [5]: each core's assigned logical qubits are placed by descending interaction-graph degree onto physical positions ordered by ascending intra-core centrality, then locally refined by a bounded pairwise-swap pass. TELESABRE is again excluded, for the same structural reason as above (no externally supplied layout support in its CLI). `HypergraphPartitioning` only decides which *core* each logical qubit goes to; the capacity it sees for a core is the length of that core's declared qubit list in the `NISQNetwork`. We withhold the same four chip-wide corner physical qubits from every core's declared capacity that DSABRE's own SabreLayout corners-removed candidates already reserve, the `pytket-dqc` analogue of that corner removal, and apply the same exclusion when placing logical qubits onto physical positions within each core. As with SabreLayout's own corner removal, this guarantees only a four-qubit margin *chip-wide*, not per-core, so a core can still end up fully packed; rather than skip the run in that case, PORTER's general safeguard against fully packed cores (above) legalizes the layout automatically before routing begins. DSABRE's teledata-only mechanism does not need a free port on the side it teleports *from* and was never affected by this precondition. Because `pytket-dqc` uses gate teleportation, not state teleportation, its own reported cost is in a different unit (e-bits per Steiner-tree-amortised hyperedge, not EPR pairs per qubit move) and is included only as context, following DSABRE's own established convention for this comparison [5].

TABLE III
25-QUBIT SUITE, OWN-PROTOCOL COMPARISON (ARCHITECTURE A, 2×2 CORES OF 3×3, 36 PHYSICAL QUBITS). BEST-OF-3(-OR-6) PER ROUTER AS DESCRIBED IN SECTION VI-A. Δ_{TS} IS PORTER'S EPR CHANGE RELATIVE TO TELESABRE; NEGATIVE IS BETTER.

	TELESABRE		DSABRE		PORTER		
Circuit	EPR	SWAP	EPR	SWAP	EPR	SWAP	Δ_{TS}
ae	118	370	94	399	87	356	−26.3
ghz	11	30	4	18	6	32	−45.5
graphstate	17	43	4	19	9	42	−47.1
qft	141	371	91	462	94	382	−33.3
qnn	357	770	155	790	159	734	−55.5
random	632	1512	431	1521	378	1311	−40.2
gmean (6)	94.3		45.7		54.6		**−42.1**

TABLE IV
64-QUBIT SUITE, OWN-PROTOCOL COMPARISON (ARCHITECTURE H, 2×3 CORES OF 4×4, 96 PHYSICAL QUBITS). METHODOLOGY AND NOTATION AS TABLE III. *TELESABRE FAILS TO CONVERGE WITHIN 3 SEEDS; GMEAN IS OVER THE 5 CIRCUITS ON WHICH IT DOES.

	TELESABRE		DSABRE		PORTER		
Circuit	EPR	SWAP	EPR	SWAP	EPR	SWAP	Δ_{TS}
ae	323	1508	244	1863	316	1970	−2.2
ghz	11	111	14	73	29	189	+163.6
graphstate	76	379	18	113	31	193	−59.2
qft	410	1872	240	2252	333	2279	−18.8
qnn	1365	6292	539	8394	642	5997	−53.0
random*	—	—	738	3827	695	3837	—
gmean (5)	172.1		95.5		143.4		**−16.7**

B. Main Results

TELESABRE's EPR count is teledata plus telegate operations; DSABRE's is teledata only; PORTER's is teleport plus telegate plus telegate-packet operations (one EPR pair per packet, regardless of how many remote CX gates it amortises). All three report in the same one-EPR-pair unit. SWAP counts intra-core SWAPs only.

25q, own protocol. PORTER reduces geometric-mean EPR consumption by **42.1%** against TELESABRE across all six circuits, with per-circuit reductions ranging from -26.3% (AE) to -55.5% (QNN) and no regressions on this suite. Against DSABRE, the picture is reversed: it beats PORTER on every circuit here, most sharply on the two structurally sparse ones (GHZ: 4 vs. 6; Graphstate: 4 vs. 9), where SabreLayout on the corners-removed architecture graph, refined over three forward–backward–forward routing passes, finds a near-ideal core partition before routing begins – exactly the regime DSABRE's own evaluation identifies as its strongest case [5]. On the three CX-dense circuits (AE, QFT, QNN) the gap to DSABRE is narrower, consistent with PORTER's TELEGATEPACKET primitive recovering some of the ground a teledata-only router gives away when many remote CX gates share one control qubit.

25q, shared layout. Under TELESABRE's own initial layout (Table VI), both DSABRE and PORTER still improve substantially on TELESABRE (-37.3% and -40.1% gmean

TABLE V
80-QUBIT SUITE, OWN-PROTOCOL COMPARISON (ARCHITECTURE H, SAME PHYSICAL TOPOLOGY AS THE 64-QUBIT SUITE, MORE LOGICAL QUBITS PER CIRCUIT). METHODOLOGY AND NOTATION AS TABLE III. AE EXCLUDED (SECTION VI-A). *TELESABRE FAILS TO CONVERGE; GMEAN IS OVER THE 3 CIRCUITS ON WHICH IT DOES.

	TELESABRE		DSABRE		PORTER		
Circuit	EPR	SWAP	EPR	SWAP	EPR	SWAP	Δ_{TS}
ghz	19	172	20	143	35	224	+84.2
graphstate	88	521	22	131	37	252	−58.0
qft*	—	—	670	3844	539	3627	—
qnn	19	172	25	146	35	224	+84.2
random*	—	—	—	—	10855	61295	—
gmean (3)	31.7		22.2		35.7		**+12.6**

TABLE VI
25-QUBIT SUITE, SHARED-LAYOUT COMPARISON: DSABRE AND PORTER ARE BOTH ROUTED, SINGLE-SHOT, FROM TELESABRE'S OWN INITIAL LAYOUT (SECTION VI-A). Δ_{TS} COLUMNS GIVE EACH ROUTER'S EPR CHANGE RELATIVE TO TELESABRE'S OWN-PROTOCOL RESULT; NEGATIVE IS BETTER.

	TELESABRE		DSABRE			PORTER		
Circuit	EPR	SWAP	EPR	SWAP	Δ_{TS}	EPR	SWAP	Δ_{TS}
ae	118	370	77	393	−34.7	62	267	−47.5
ghz	11	30	8	33	−27.3	6	26	−45.5
graphstate	17	43	10	37	−41.2	14	57	−17.6
qft	141	371	89	410	−36.9	96	343	−31.9
qnn	357	770	177	818	−50.4	158	707	−55.7
random	632	1512	439	1529	−30.5	412	1392	−34.8
gmean (6)	94.3		59.1		**−37.3**	56.5		**−40.1**

TABLE VII
64-QUBIT SUITE, SHARED-LAYOUT COMPARISON. METHODOLOGY AND NOTATION AS TABLE VI. *TELESABRE FAILS TO CONVERGE; GMEAN IS OVER THE 5 CIRCUITS ON WHICH IT DOES.

	TELESABRE		DSABRE			PORTER		
Circuit	EPR	SWAP	EPR	SWAP	Δ_{TS}	EPR	SWAP	Δ_{TS}
ae	323	1508	183	1963	−43.3	233	1753	−27.9
ghz	11	111	8	124	−27.3	8	104	−27.3
graphstate	76	379	32	167	−57.9	49	289	−35.5
qft	410	1872	228	2104	−44.4	227	1668	−44.6
qnn	1365	6292	544	8250	−60.1	679	5963	−50.3
random*	—	—	—	—	—	—	—	—
gmean (5)	172.1		89.7		**−47.9**	107.1		**−37.8**

TABLE VIII
80-QUBIT SUITE, SHARED-LAYOUT COMPARISON. METHODOLOGY AND NOTATION AS TABLE VI. *TELESABRE FAILS TO CONVERGE (NO INITIAL LAYOUT TO EXTRACT); GMEAN IS OVER THE 3 CIRCUITS ON WHICH IT DOES.

	TELESABRE		DSABRE			PORTER		
Circuit	EPR	SWAP	EPR	SWAP	Δ_{TS}	EPR	SWAP	Δ_{TS}
ghz	19	172	15	173	−21.1	11	156	−42.1
graphstate	88	521	55	306	−37.5	71	402	−19.3
qft*	—	—	—	—	—	—	—	—
qnn	19	172	15	173	−21.1	11	156	−42.1
random*	—	—	—	—	—	—	—	—
gmean (3)	31.7		23.1		**−27.0**	20.5		**−35.3**

respectively), and PORTER is now the better of the two on four of six circuits (AE, GHZ, QNN, Random) and on the gmean overall, reversing the own-protocol ranking. DSABRE still wins on the two circuits where it wins most decisively under its own protocol too (Graphstate, QFT). This isolates the effect seen in the own-protocol table: on this suite, a meaningful share of DSABRE's own-protocol advantage over PORTER traces to SabreLayout's stronger initial partition rather than to routing-time decisions alone – when both routers start from the identical (weaker, TeleSABRE-optimised-for-itself) layout, PORTER edges ahead on four of six circuits and on the suite gmean, though DSABRE retains an edge specifically on the circuits its partition-first strategy suits best.

64q, own protocol. TELESABRE fails to converge within three seeds only on Random here; on QNN (8,126 CX) it succeeds at 1,365 EPR pairs and 6,292 SWAPs, exactly matching DSABRE's own published number for this circuit [5] – a useful cross-check that our harness reproduces TELESABRE faithfully. (An earlier version of this evaluation reported QNN as non-convergent; that was a bug in our own harness, not TELESABRE – see Section VI-C.) The gmean row is computed over the five circuits on which TELESABRE converges. PORTER beats TELESABRE on AE, Graphstate, QFT, and QNN (−2.2% to −59.2%), but GHZ regresses sharply (+163.6%, 29 vs. 11 EPR pairs). This is not unique to PORTER: DSABRE regresses on the same circuit in our reproduction (14 vs. 11, +27.3%) and in DSABRE's own published 64q table (+45.5%) [5]. GHZ's circuit structure – every front gate shares one control qubit, so a per-gate/per-candidate local scorer chases the *next* CX in the chain – defeats locally-greedy schedulers generally, not any one design; TELESABRE's front-averaged energy happens to find a better global rendezvous core on this specific pathological case. On the one circuit TELESABRE fails to complete (Random), PORTER does (695 EPR), without any deadlock-recovery mechanism firing.

64q, shared layout. TELESABRE's initial layout is now available for five of six circuits (Table VII). DSABRE wins the gmean here (−47.9% vs. PORTER's −37.8%), reversing the 25q shared-layout ranking: it wins AE, Graphstate, and QNN outright, ties PORTER on GHZ (both −27.3%), and is edged out only narrowly on QFT (228 vs. 227 EPR). This is the opposite pattern from the own-protocol table, where PORTER beats DSABRE on AE, QFT, and QNN; on architecture H's larger, 16-slot cores, DSABRE's forward–backward–forward routing passes evidently extract more value from a fixed layout than PORTER's single forward pass does, even though PORTER's own layout search (which additionally varies the partition itself, not just the route) still comes out ahead on AE, QFT, and QNN overall (Table IV). Taken together, the two suites suggest the layout-versus-routing split of DSABRE's advantage is architecture-dependent rather than a fixed property of either router.

80q. TELESABRE converges on only three of five circuits here (Tables V and VIII); GHZ and QNN are numerically identical because the installed `mqt.bench`'s QNN generator

TABLE IX
PYTKET-MAPPING SHARED-LAYOUT COMPARISON (25Q AND 64Q SUITES; SECTION VI-A), CORNERS-REMOVED NISQNETWORK CAPACITY. PYTKET E-BITS IS HYPERGRAPHPARTITIONING'S OWN COST IN A DIFFERENT UNIT (CONTEXT ONLY, NOT EPR-COMPARABLE). PORTER'S GENERAL FULL-CORE SAFEGUARD (SECTION VI-A) LEAVES NO MISSING ENTRIES ON THIS SUITE.

Suite	Circuit	pytket e-bits	DSABRE		PORTER	
			EPR	SWAP	EPR	SWAP
25q	ae	78	67	339	73	348
25q	ghz	3	3	14	3	17
25q	graphstate	4	5	24	4	19
25q	qft	70	92	437	118	438
25q	qnn	148	168	848	127	703
25q	random	564	440	1482	395	1340
64q	ae	264	361	2243	223	1638
64q	ghz	5	5	74	5	62
64q	graphstate	10	8	68	10	60
64q	qft	307	320	2112	278	2021
64q	qnn	621	509	8215	1031	6538
64q	random	1020	750	3969	861	4384

produces a near-linear circuit at this size (Section VI-A), so both are effectively the same chain-structured pathological case discussed above, and PORTER regresses on both by the same $+84.2\%$ under its own protocol. The striking result is on the two circuits TELESABRE cannot complete: on Random (23,381 CX, by far the largest circuit in any suite here), DSABRE's own-protocol run also exceeds our uniform 900 s per-call compiler timeout (Section VI-A) – recorded as a missing entry, the same as TELESABRE's – but PORTER completes it in 40 s at 10,855 EPR pairs. Running DSABRE past that cap specifically to characterize its failure mode (rather than as part of the standard evaluation) shows it is not a deadlock: by the time it stops it has already placed 7,713 EPR pairs and 41,421 SWAPs, genuine forward progress. DSABRE's `HardwareConfig` for this suite (`max_iterations=20000`, carried over unchanged from the 64-qubit suite, per DSABRE's own scaling guidance to increase it for larger circuits [5]) is reached after 938.4 s of its first forward pass alone, having cleared only 1,233 of the circuit's 23,381 CX gates; the full own-protocol search evaluates up to three layout candidates through up to three such passes each, so the practical cost of simply raising the budget far enough to finish this circuit would likely run into hours, well beyond our uniform timeout. This is a resource-budget limitation specific to this unusually dense circuit (23,381 CX over 80 qubits, roughly 3–18× the CX/qubit ratio of any other circuit in our suites), not evidence that DSABRE's routing algorithm cannot in principle handle it. On QFT (2,660 CX) both DSABRE and PORTER complete where TELESABRE cannot, with PORTER using 19.6% fewer EPR pairs than DSABRE (539 vs. 670). We read this cautiously – one non-AE-capable suite with reduced circuit diversity is not conclusive – but it is consistent with the 64q finding that PORTER tends to complete circuits at scales where TELESABRE, and sometimes DSABRE under a fixed iteration budget, do not.

Pytket-mapping comparison. Table IX now reports a complete grid: PORTER's full-core safeguard (Section VI-A) eliminates the three missing entries an earlier version of this evaluation reported for PORTER on this protocol (25q QNN, 64q AE, 64q QNN), at the cost of the layout-quality hit an emergency eviction inevitably takes – most visibly on 64q QNN, where the evicted layout lands PORTER at 1,031 EPR pairs against DSABRE's 509, its worst relative result on this suite. Excluding that outlier, results split close to evenly: PORTER uses fewer EPR pairs on 25q Graphstate/QNN/Random and 64q AE/QFT, DSABRE on 25q AE/QFT and 64q Graphstate/Random, and the two tie on both GHZ rows; geometric-mean EPR across all twelve rows is within 1.4% of parity (DSABRE 69.2 vs. PORTER 70.2). `pytket-dqc`'s own e-bit cost is in a structurally different unit (Steiner-tree-amortised gate teleportation, not per-qubit state teleportation) and is not directly comparable, per DSABRE's own established convention for this comparison [5]; we include it only as context.

C. The TELESABRE Filename-Handling Bug

Our comparison harness originally invoked TELESABRE with the 64-qubit QNN circuit file's on-disk name, `qnn_OLD_DEEP....qasm.bak` (the deeper of two QNN variants present in the upstream repository, selected because its CX count exactly matches DSABRE's own published QNN table). TELESABRE's `main.c` infers file type from the *last* dot in the filename, so this double extension is parsed as `.bak` and rejected outright (`"does not have a .json or .qasm extension"`) before any routing is attempted – a harness bug, not a genuine TELESABRE non-convergence. We fixed this by always handing the binary a temporary copy with a clean `.qasm` suffix. Once fixed, TELESABRE completes this circuit in 21.7 s at exactly the EPR and SWAP counts DSABRE's own evaluation reports (Section VI-B), confirming the fix and giving an independent cross-check of our harness's fidelity to TELESABRE's actual behaviour. All TELESABRE numbers in this paper reflect the corrected harness.

D. The TELESABRE Reference-Implementation Bug

Running TELESABRE's compiled C binary against architecture A (four $3\times3=9$-qubit cores) with its default hungarian initial-layout configuration segfaults deterministically on every circuit we tried. We traced this with source-level instrumentation (macOS's `lldb` could not attach headlessly in our environment) to `initial_layout_hungarian` in `src/layout.c`: its second assignment pass places a logical qubit q into the first core whose remaining capacity exceeds `init_layout_hun_min_free_qubit` (4 in the reference config), but if *every* core's remaining capacity is at or below that threshold – which routinely happens once a 9-slot core has already absorbed 5 of its share of a 25-logical-qubit circuit – `virt_to_core[q]` is left at its sentinel -1 and is then used, unchecked, as a negative array index a few lines later. This is a latent bug in the reference implementation, not an artefact of our harness: it reproduces with the project's own

TABLE X
PER-ATTEMPT WALL-CLOCK TIME (SECONDS) ON REPRESENTATIVE CIRCUITS; ONE CANDIDATE LAYOUT/SEED PER ROUTER. *DID NOT COMPLETE.

Circuit	CX	TELESABRE	DSABRE	PORTER
ae (25q)	558	0.40	1.39	0.13
random (25q)	1124	1.30	5.65	0.42
ae (64q)	1962	0.99	30.90	0.40
qft (64q)	1966	1.99	14.03	0.39
qnn (64q)	8126	21.70	137.88	2.04
random (80q)	23381	*	*	40.14

`configs/default.json` and device file, invoked exactly per the project's README. It is specific to core capacities materially smaller than the $4\times4{=}16$ cores TELESABRE was originally evaluated on; we did not observe it on architecture H. We built a minimal, behaviour-preserving patch (fall back to the least-loaded core rather than leaving the qubit unassigned) against a separate copy of the source tree and used the resulting binary for every TELESABRE number in this paper; the project's own repository is untouched.

E. Compile Time

Table X reports per-attempt wall-clock time: one `route` call for DSABRE (one SabreLayout candidate through its forward–backward–forward pass schedule), one `telesabre_run` attempt for TELESABRE, and one `compile` call for PORTER, all on the same machine. Search cost scales with the number of candidates each router tries (three for TELESABRE/DSABRE, six for PORTER, Section VI-A), so the totals in Section VI-B are correspondingly 3–6$\times$ these per-attempt figures. PORTER is consistently the fastest per attempt, and the gap widens with circuit size: on the 1,962-CX AE circuit it is 77$\times$ faster than DSABRE; on the 8,126-CX QNN circuit, 68$\times$; on the 23,381-CX 80-qubit Random circuit, DSABRE's own-protocol attempt does not complete at all (Table V) while PORTER finishes in 40 s. This reflects the performance engineering of Section V as much as any algorithmic difference between the routers – DSABRE's reference implementation has not been subjected to the same optimisation pass – so it should be read as a statement about implementation maturity rather than an inherent complexity gap between the two approaches.

VII. RELATED WORK

TELESABRE [6] is our closest architectural relative: both routers support teledata and gate teleportation and both build on the SABRE extended-set scoring idea. The two differ chiefly in decision structure: TELESABRE scores inter-core candidates via a contracted-communication-graph Dijkstra energy that mixes staging cost and lookahead into one flat action pool per candidate, whereas PORTER scores remote *plans* independently of their preparation cost and only generates preparation SWAPs for the one plan selected (Section III-C), and adds a same-control packet primitive that amortises runs of shared-control remote CX gates onto one EPR pair. PORTER's capacity handling is also more proactive: alongside TELESABRE's safety-valve iteration cap, PORTER maintains an explicit blocked-plan set and both a breadth-first tail-relief search and an opportunistic pre-emptive relief comparator (Section III-G).

DSABRE [5] (we compare against its default `dSABRE_BurstExt` variant, "DSABRE") is the more EPR-efficient baseline under each router's own protocol (Section VI-B), particularly on sparse, highly partitionable circuits (GHZ, Graphstate) where its SabreLayout-based initial allocation plus forward-backward-forward routing passes find a near-ideal core partition before routing even begins. The shared-layout comparison (Tables VI, VII, and IX) attributes a substantial share of that advantage to initial-layout quality rather than routing-time decisions on the 25-qubit suite – from TELESABRE's own layout, PORTER matches or beats DSABRE on four of six circuits – though the effect reverses on the 64-qubit suite (Section VI-B). DSABRE restricts itself to teledata; PORTER instead follows TELESABRE in also supporting gate teleportation and extends it with same-control packets, which matters on circuits with long runs of shared-control remote CX gates (e.g. QFT-like phase-kickback patterns) where amortising many remote CX gates onto one EPR pair can outweigh a less-optimal core partition.

Single-QPU qubit mapping. The front+extended-set scoring loop PORTER and both baselines build on originates with SABRE [11]; LIGHTSABRE [14] formalises the Δ-form SWAP score PORTER reuses for its intra-core stage (Eq. 1) and inter-core direct-gain term (Eq. 3).

Allocation and scheduling. Hypergraph-partitioning allocation methods [13], [15] and time-aware schedulers [16], [17] address the upstream allocation and downstream scheduling stages respectively; both are complementary to PORTER, which addresses only the routing stage given a fixed initial layout.

VIII. CONCLUSION

We presented PORTER, a plan-committed router for multi-core distributed quantum computers whose central design choice is to separate remote-plan *selection* from remote-plan *preparation*, in contrast to TELESABRE and DSABRE, which score inter-core candidates and their staging SWAPs together in one flat action pool: every legal teleport, telegate, and same-control packet candidate (including capacity-blocked variants) is scored on its physical and DAG-lookahead merits before any core-preparation SWAP is generated, and preparation effort is then spent exclusively on the one plan chosen. Combined with a two-tier capacity-relief mechanism (breadth-first tail relief when every plan is blocked, opportunistic pre-emptive relief when one is severely blocked), this plan-first structure lets PORTER reduce geometric-mean EPR consumption by 42.1% against TELESABRE on a 25-qubit suite and 16.7% on a 64-qubit suite, completing every circuit TELESABRE cannot, while remaining behind DSABRE under each router's own best-practice layout, particularly on sparse, easily-partitioned circuits and on the chain-structured GHZ pattern where all locally-greedy schedulers we tested,

PORTER included, lose ground to TeleSABRE's front-averaged energy. Feeding both routers TeleSABRE's own initial layout instead of each one's own best-practice layout reverses this ranking on the 25-qubit suite but not on the 64-qubit one, indicating that the share of dSABRE's advantage attributable to its initial-layout heuristic versus its routing-time decisions is architecture-dependent – a distinction that matters for anyone choosing where to invest further engineering effort between the two stages. At the largest scale we tested (80 qubits, a 23,381-CX circuit), PORTER completed in 40 s where both TeleSABRE and dSABRE exceed our uniform 900 s per-call compiler timeout; running dSABRE past that cap (Section VI-B) shows 938.4 s of genuine forward progress toward its own iteration budget rather than a deadlock – a practical throughput advantage on unusually dense circuits at scale that the smaller suites do not exercise, though not evidence that dSABRE's algorithm is inherently unable to route this circuit given a larger budget. We verified the reference implementation with an independent replay checker across 60 randomized circuits and the full evaluation suite, applied a set of behaviour-preserving performance fixes yielding a $4\times$ compile-time speedup with byte-identical output, and, in the course of building a fair three-way comparison, found and patched a latent out-of-bounds bug in TeleSABRE's reference C implementation that had prevented it from running at all on small-core architectures, and separately identified and fixed a filename-handling bug in our own harness that had caused an earlier version of this evaluation to under-report TeleSABRE's 64-qubit convergence.

Artefact availability.: The PORTER implementation, the independent replay verifier, the cross-router comparison harness, and raw per-circuit JSON results are available in the accompanying repository.

Acknowledgments

This work was partially supported by the Natural Science Foundation of China (Grant No.). The author used Anthropic's Claude (via Claude Code) to assist with: (i) reviewing and applying behaviour-preserving performance fixes to the PORTER implementation, (ii) authoring the independent replay verifier and the cross-router comparison harness, (iii) diagnosing and patching the TeleSABRE reference-implementation bug described in Section VI-D, and (iv) drafting the LaTeX source of this manuscript. All AI-generated content was reviewed and verified by the author, who takes full responsibility for the correctness and originality of the work.

References

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[4] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[5] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.

[6] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[7] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[8] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.

[9] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.

[10] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[11] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[12] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[13] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[14] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, "LightSABRE: A lightweight and enhanced SABRE algorithm," 2024. [Online]. Available: https://arxiv.org/abs/2409.08368

[15] P. Andres-Martinez and C. Heunen, "Automated distribution of quantum circuits via hypergraph partitioning," *Physical Review A*, vol. 100, no. 3, p. 032308, 2019.

[16] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.

[17] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.



Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.